

Audit Agent et feuille de route

Analyse du dépôt côté Agent · synthèse destinée à une IA d'implémentation

Périmètre : runtime, outils, API, planification, persistance et interface associée. Sources limitées au dépôt et aux tests locaux. Revue statique au 23 septembre 2026 ; aucune modification du code ni exécution de tests.

Conclusion — la base Agent possède déjà des protections solides (approbations persistées, contrôle d'accès côté serveur, budget cumulé, fetch SSRF défensif). Trois écarts méritent toutefois une correction prioritaire : capacités incohérentes en exécution planifiée lors d'un fallback modèle, absence apparente d'un claim atomique du run interactif, et validation média trop large dans la route.

Décision proposée

- P0 : corriger le fallback de modèle planifié et empêcher deux POST concurrents de créer deux runs actifs pour un même chat.
- P1 : brancher la validation MIME/capacités existante dans la route et rendre explicite le timeout du tick de scheduler.
- P2 : compléter l'observabilité, le parcours de reprise et l'évaluation continue avant ouverture plus large de l'Alpha.

PARCOURS DE REMÉDIATION AGENT

Ordre conseillé pour rendre les exécutions fiables, prévisibles et pilotables

01 P0 · FIABILITÉ

Corriger maintenant

- Aligner le modèle planifié
- retenu et ses capacités
- Verrouiller atomiquement
- la création/reprise du run

02 P1 · GARDE-FOUS

Durcir le parcours

- Valider les types de fichiers
- contre les capacités du modèle
- Borner chaque tick cron
- et traiter les expirations

03 P2 · ÉVOLUTION

Améliorer ensuite

- Observabilité des checkpoints
- et des décisions de scheduler
- Reprise manuelle guidée
- et historique des versions
- Eval qualité / coût / succès

P0 = correction prioritaire · P1 = avant élargissement Alpha · P2 = feuille de route

Figure 1. Séquence de mise en œuvre proposée.

Constats à corriger

1. Le fallback planifié conserve les capacités du modèle demandé

P0 · Confiance Élevée

Preuve — Dans `lib/agent/scheduler/execute.ts`, le modèle demandé est chargé en `modelEntry` puis `resolvedModelId` peut être remplacé. La sélection des outils et le contexte utilisent encore `modelEntry.capabilities` (lignes 211–259 et 290–296 dans cette zone).

Impact — Un modèle demandé sans outils peut provoquer une sélection incohérente après fallback : capacités de contexte erronées, et filtrage des outils fondé sur le mauvais modèle. La tâche planifiée peut échouer ou s'exécuter avec un contexte inadapté.

Correction — Après résolution, charger `resolvedEntry = getModelEntry(resolvedModelId, FALLBACK_MODELS)`, puis utiliser `resolvedEntry` partout : sélection, contexte, limites médias et création du run.

Critère d'acceptation — Un test de contrat force un modèle sans outils puis vérifie que le modèle de repli et ses capacités gouvernent la sélection et le contexte.

2. La création d'un run interactif n'est pas atomique

P0 · Confiance Élevée

Preuve — `app/(chat)/api/agent/route.ts` recherche un run actif puis appelle `createAgentRun` dans un chemin séparé (zone lignes 217–365). `getActiveAgentRunByChatId` effectue un `SELECT` ; le schéma `AgentRun` ne déclare que des index ordinaires sur `chatId` et `statut` (`lib/db/agent-queries.ts:105–125` ; `lib/db/schema.ts:1151–1158`).

Impact — Deux requêtes concurrentes sur le même chat peuvent toutes deux voir l'absence de run actif et créer chacune un `AgentRun`. L'interface et la persistance peuvent alors diverger et les actions/outils être exécutés en double. Le chemin planifié possède un mécanisme de `lease`, mais le chemin interactif ne montre pas de claim équivalent.

Correction — Introduire une réservation atomique par chat (transaction/verrou ou table de claim avec contrainte unique), rendre `create-or-resume` idempotent et renvoyer le run déjà réclamé à la requête perdante.

Critère d'acceptation — Un test concurrent lance deux créations simultanées et observe un seul run actif et une seule exécution d'outil.

3. Les types de fichiers ne sont pas validés contre les capacités détaillées

P1 · Confiance Élevée

Preuve — La route valide uniquement une capacité globale et le nombre de fichiers (`app/(chat)/api/agent/route.ts:183–201`). `validateAttachmentsAgainstModel` existe dans `lib/agent/context/files.ts:46–88`, mais rg n'a relevé aucun appelant. Le schéma accepte des préfixes `image/*` et `text/*` (`app/(chat)/api/agent/schema.ts:15–40`).

Impact — Un média accepté à l'entrée peut ne pas être pris en charge par le modèle (images/documents), ou être rejeté seulement plus tard lors de la lecture. L'utilisateur obtient alors une erreur tardive et le run dépense des tokens/outils inutilement.

Correction — Appeler `validateAttachmentsAgainstModel` immédiatement après la résolution effective du modèle ; resserrer si besoin les MIME déclarés et vérifier le `type/contenu` téléchargé avant extraction.

Critère d'acceptation — Tests API couvrant un modèle sans vision avec image, sans documents avec PDF, type non pris en charge et média valide ; refus explicite avant création du run.

4. Le timeout déclaré du scheduler n'est jamais appliqué

P1 · Confiance Élevée

Preuve — `lib/agent/scheduler/engine.ts:266–270` déclare `TICK_RUN_TIMEOUT_MS`, mais aucune référence à cette constante n'existe ailleurs dans le fichier. La boucle `runSchedulerTick` traite les `schedules` séquentiellement (lignes 287–302). Le commentaire promet un tick borné, mais aucun `AbortSignal/Promise.race` ne borne `processDueSchedule` ici.

Impact — Un appel fournisseur ou un run bloqué peut retenir un worker au-delà de la durée prévue ; avec plusieurs échéances séquentielles, le retard se propage aux tâches suivantes. La `lease` et les reprises réduisent les doublons, mais ne remplacent pas une limite d'exécution du worker.

Correction — Appliquer une deadline réelle au tick/run, propager l'annulation jusque dans le runtime et conserver le checkpoint lors de l'expiration. Définir aussi le comportement des `schedules` en attente de résolution, distinct d'un timeout d'exécution.

Critère d'acceptation — Test avec exécution simulée non résolue : le tick termine avant sa deadline, enregistre l'état attendu et la prochaine exécution reprend le même run sans doublon.

Améliorations recommandées

Observabilité exploitable

Remplacer les catch silencieux des points critiques par des événements structurés sans secrets : checkpoint non persisté, compteur non mis à jour, échec de `lease`, reprise et décision de timeout. Ajouter des métriques sur durée active, tentatives, attente d'approbation et âge des occurrences.

Reprise explicable

Dans la timeline, afficher le motif de suspension, les éléments repris et la limite restante ; proposer une action « reprendre » après timeout, en réutilisant l'identité de run et son checkpoint.

Contrats de capacités partagés

Centraliser la résolution modèle→capacités pour les chemins interactif et planifié afin d'éviter que le modèle demandé et le modèle effectif divergent. Ajouter des tests de parité aux tests unitaires de sélection.

Évaluations de bout en bout

Créer un petit jeu de scénarios déterministes : recherche web avec sources, lecture de fichier, approbation accordée/refusée, réponse à une question, interruption, reprise, budget épuisé et scheduler en échec. Mesurer réussite, durée et coût sans dépendre d'un seul fournisseur.

Nouvelles fonctionnalités à planifier

Fonctionnalité	Valeur utilisateur	Prérequis / garde-fou
Mode prévisualisation des actions	Voir les outils, paramètres et effets prévus avant lancement.	S'appuyer sur le snapshot de permissions et les empreintes d'approbation.
Reprise avec choix guidé	Reprendre une tâche interrompue, réduire son périmètre ou modifier ses consignes.	Verrou de run atomique, checkpoint fiable et budget restant visible.
Historique/versioning des tâches planifiées	Comparer consigne, modèle, outils et résultats entre occurrences.	Journal de changements versionné et politique de rétention.
Évaluation qualité et coût par tâche	Comparer modèles/réglages sur critères de réussite définis.	Télémétrie minimisée, opt-in et coût calculé avec unités explicites.

Plan d'implémentation pour l'IA

- 1. Cartographier les tests et les contrats existants ; ne pas modifier les règles d'approbation persistées.
- 2. Corriger le chemin scheduler pour utiliser la fiche du modèle résolu, puis ajouter le test de fallback.
- 3. Ajouter le claim interactif atomique ; migrer les index/contraintes avec une migration réversible et traiter d'abord les éventuels runs actifs historiques.
- 4. Brancher la validation média par capacités et couvrir les MIME limites.
- 5. Appliquer une deadline effective au scheduler et préciser la reprise en attente d'approbation/réponse.
- 6. Ajouter observabilité et critères d'acceptation, puis exécuter les tests ciblés et la suite existante.

Points déjà bien couverts

- Approbations liées au toolCallId et invalidées si les paramètres présentés changent : lib/agent/approvals/incoming.ts et tests/unit/agent-approvals.test.ts.
- Limites techniques et limites produit séparées, avec compteurs cumulatifs de reprises : lib/agent/budget.ts, lib/agent/limits.ts et tests/unit/agent-budget.test.ts.
- Téléchargements externes bornés, redirections revalidées et destinations DNS épinglées : lib/web/safe-fetch.ts et lib/agent/tools/internal/extract.ts.
- Planification avec fuseau IANA, occurrence réservée par lease et conversation dédiée au schedule : lib/agent/scheduler/engine.ts et lib/agent/scheduler/execute.ts.

Limites de cette revue

Audit statique ciblé, sans exécution de tests, sans accès à un environnement de production et sans validation des comportements des fournisseurs. Le défaut de résolution des capacités planifiées et l'absence d'appel de la validation média sont directement étayés par le code. Le risque de doublons interactifs doit être confirmé par un test concurrent et par l'analyse du niveau d'isolation PostgreSQL utilisé en production. Le timeout du scheduler est classé comme défaut manifeste d'application du contrat/commentaire, son impact exact dépend du runtime de déploiement et des délais du fournisseur.

Contrats techniques à préserver

Cette section transforme les constats en contraintes de conception. Elle aide l'IA d'implémentation à corriger les écarts sans affaiblir les protections déjà présentes.

Résolution du modèle planifié

- Calculer une seule fois l'identifiant effectif après application du fallback. La fiche effective doit alimenter le constructeur de modèle, le sélecteur d'outils, la fenêtre de contexte, les capacités de fichiers et les métadonnées persistées du run.
- Ne pas utiliser le modèle demandé pour configurer le run après fallback. Les traces doivent conserver les deux identifiants (demandé et effectif) afin que l'opérateur comprenne pourquoi le modèle a changé.
- Exercer les cas : modèle demandé absent du catalogue, modèle présent sans tools, modèle effectif avec contexte réduit et modèle par défaut indisponible. Chaque cas doit échouer explicitement ou produire un run cohérent.

Réservation interactive d'un run

- La lecture « run actif ? » suivie d'une insertion n'est pas un verrou. Choisir un mécanisme atomique compatible avec PostgreSQL et le statut multi-état ; un index unique partiel doit être étudié au regard des transitions `waiting_for_user` / `waiting_for_approval`.
- Faire retourner au client un identifiant de run stable pour les répétitions de requête. Distinguer l'idempotence d'un retry réseau de la création d'une nouvelle tâche volontaire.
- La requête perdante doit rejoindre le run existant ou recevoir une réponse de conflit explicite. Elle ne doit jamais lancer parallèlement une seconde boucle d'outils.

Contrat de validation des pièces jointes

- La validation doit être exécutée côté serveur après résolution des capacités effectives et avant le calcul du contexte, la création du run et tout appel fournisseur.
- Le MIME fourni par le client est une indication, pas une preuve du contenu. Maintenir les garde-fous de lecture déjà présents : plafond d'octets, type/contenu cohérent, ZIP guard pour DOCX et fetch réseau défensif.
- Les erreurs doivent nommer le type ou la capacité manquante, sans exposer l'URL interne ni les détails d'accès.

Validation et critères de livraison

Tests ciblés à ajouter

Zone	Scénario	Résultat attendu
Scheduler/modèle	Modèle choisi sans outils, fallback avec outils	Sélection, contexte et run reposent tous sur le modèle effectif.
Scheduler/modèle	Modèle de fallback absent ou invalide	Erreur exploitable avant création du run ; aucune tâche silencieusement dégradée.
API/race	Deux requêtes POST même chat lancées simultanément	Un seul run actif et un seul ensemble d'exécutions d'outils.
API/idempotence	Retry de la même requête après perte de réponse HTTP	Même run rendu, sans nouvel effet externe.
Fichiers	PDF envoyé à un modèle sans capacité documents	Refus côté API avant run et avant appel du fournisseur.
Fichiers	Image envoyée à un modèle sans vision	Refus explicite côté API ; aucune donnée d'image transmise.
Fichiers	MIME incohérent avec le contenu téléchargé	Échec contrôlé et borné par les protections d'extraction existantes.
Scheduler/timeout	Exécution dépasse la durée maximale du tick	Annulation propagée, checkpoint conservé, reprise idempotente.
Scheduler/reprise	Run en attente d'approbation dépasse la lease	Politique explicite : attendre résolution ou expirer selon une durée métier ; pas de boucle cron infinie.

Séquence de livraison

- Lot A — correction locale du modèle effectif dans execute.ts et tests de contrat ; faible rayon d'impact, pas de migration.
- Lot B — décision de réservation du run interactif : analyser les statuts actifs et la stratégie d'isolation, ajouter les tests de concurrence, puis seulement appliquer la migration si elle est nécessaire.
- Lot C — validation MIME dans la route et tests de capacités ; préserver les modèles multimodaux valides.
- Lot D — deadline de tick et politique de lease pour les états en attente ; instrumenter le résultat final et les reprises.
- Lot E — observabilité et fonctionnalités Alpha, chacune derrière un flag si elle touche aux effets externes.

Définition de terminé

Chaque correctif doit inclure un test déterministe du défaut, une vérification des scénarios existants d'approbation et de reprise, une migration vérifiée si le schéma change, et une trace d'observabilité qui permet de diagnostiquer le résultat en production sans journaliser de secret ou de contenu privé. La revue doit confirmer qu'aucun chemin interactif ou planifié ne continue à utiliser une fiche de modèle obsolète.